

International Islamic University Chittagong

Department of Computer Science and Engineering

Emergency Resource Allocator

A C++ Prioritization Simulator Using Heaps, Hash Tables,
Queues, Linked Lists, Trees, and Graphs

Submitted by

Md. Najmul Hasan Shihab ID: C223106

Miskatul Islam Chowdhury ID: C223082

Mohammad Habibul Hasan Showkat ID: C223087R

Supervised by

Tanvir Mahtab Zihan
Lecturer, Dept. of CSE, IIUC

August 4, 2026

Abstract

The Emergency Resource Allocator is a C++ triage simulator that models how a hospital or disaster-response unit decides which case to treat first and which resource to assign to it. Incoming emergencies are prioritised with a max-heap, resources are indexed by identifier in a hash table, and cases that cannot be served immediately are preserved in a FIFO waiting queue and retried when a resource is released. The base triage core is extended with a linked-list patient registry that tracks status transitions, a binary search tree that provides an ordered view of the resource pool, and a weighted graph that suggests a shortest route from a patient's location to a compatible resource.

The system exposes the same allocator state through two interfaces: a menu-driven console program and a browser dashboard served by the C++ backend. This report presents the problem statement, design decisions, module structure, core algorithms, complexity analysis, test results, and known limitations of the implementation.

Contents

Abstract	i
1 Introduction	1
1.1 Problem Statement	1
1.2 Objectives	1
1.3 Scope and Assumptions	2
2 System Design	3
2.1 Architecture	3
2.2 Triage Workflow	4
3 Data Structures	5
3.1 Max-Heap: Emergency Prioritisation	5
3.2 Hash Table: Resource Management	5
3.3 Queue: Overflow Handling	5
3.4 Linked List: Patient Registry	6
3.5 Binary Search Tree: Ordered Resource Index	6
3.6 Weighted Graph: Route Planning	6
4 Implementation	7
4.1 Emergency Representation	7
4.2 Dispatch Algorithm	7
4.3 Release and Retry	8
4.4 Route Computation	8
4.5 Interfaces	8
5 Complexity Analysis	11
6 Testing and Results	12

6.1	Build Verification	12
6.2	Functional Test Cases	13
6.3	Demo Scenario	13
7	Limitations and Future Work	15
8	Conclusion	16
	References	17

1. Introduction

Emergency response systems must decide quickly which case to serve first, which resource to deploy, and what to do when demand exceeds supply. These decisions are made under time pressure and with incomplete information, and the ordering rules matter: serving cases in arrival order can cost lives, while serving purely by severity can starve older low-severity cases indefinitely.

This project models that decision process in software. It is deliberately small enough to be read end to end, but large enough that several classical data structures must cooperate within a single workflow rather than being demonstrated in isolation. The implementation began as a triage core built on a heap, a hash table, and a queue, and was later extended with a patient registry, an ordered resource index, and a route planner.

1.1 Problem Statement

Design and implement a system that can:

- accept incoming emergencies described by severity, type, required resource, arrival time, and location;
- always identify and serve the most urgent outstanding case, without starving equally urgent older cases;
- locate and reserve a compatible resource in near-constant time;
- preserve, rather than discard, cases that arrive when no compatible resource is free, and retry them automatically when supply returns;
- maintain an auditable record of allocations and patient status changes;
- suggest a travel route between a patient and an assigned resource; and
- make all of the above inspectable through both a console and a browser interface.

1.2 Objectives

1. Apply a max-heap, hash table, queue, linked list, binary search tree, and weighted graph to a single coherent problem.

2. Justify each structure by the operations it must support, rather than by novelty.
3. Keep the allocator logic in one service layer so that both interfaces share identical behaviour.
4. Provide a repeatable demo scenario suitable for evaluation and defence.

1.3 Scope and Assumptions

The project is a simulation, not a medical-grade system. It operates on a single allocator instance held in memory, with no persistence, authentication, or coordination across facilities. Severity is supplied by the operator rather than inferred, travel cost is a static edge weight with no traffic model, and resource compatibility is a simple type match.

2. System Design

The system separates allocator state and logic from presentation. All behaviour lives in a single service class in C++; both the console program and the web backend are thin clients over that service, which guarantees that the two interfaces cannot drift apart.

Component	Responsibility
AllocatorService	Owens all state; coordinates every operation
src/main.cpp	Console interface; menu-driven interaction
src/web_main.cpp	HTTP backend; serves the UI and JSON state
ui/	Browser dashboard (HTML, CSS, JavaScript)

Table 2.1: Top-level components of the project.

2.1 Architecture

Figure 2.1 shows how the service mediates between the interfaces and the data-structure layer. No interface touches a data structure directly.

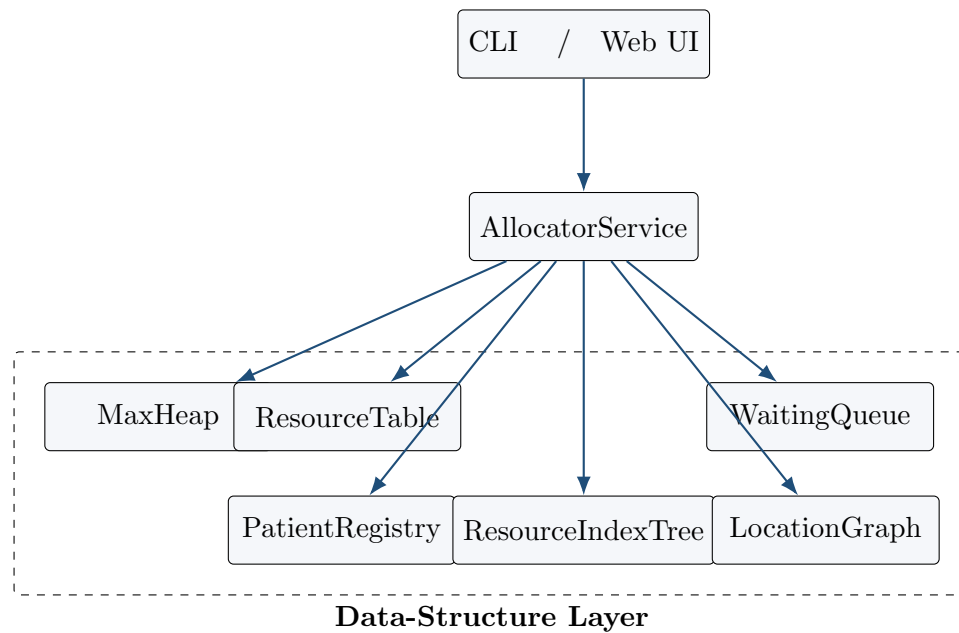


Figure 2.1: High-level architecture. Both interfaces share one service layer.

2.2 Triage Workflow

Figure 2.2 traces a single emergency from arrival to assignment or deferral.

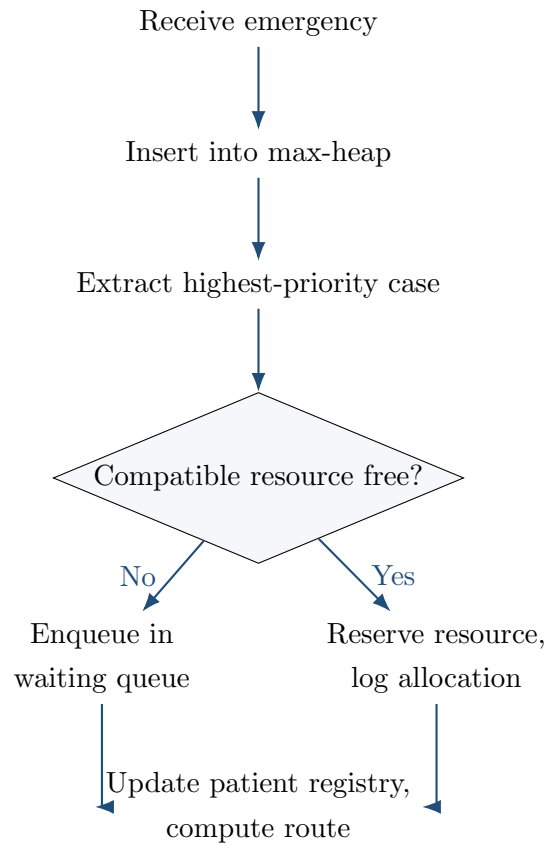


Figure 2.2: Core triage flow executed for each dispatch.

3. Data Structures

Each structure was selected for the operation that dominates its use, and the chapter states that operation explicitly rather than listing generic advantages.

3.1 Max-Heap: Emergency Prioritisation

Emergencies are stored in an array-backed binary max-heap ordered by severity, with arrival time as a tie-breaker so that equally severe cases are served in arrival order and cannot be starved. The dominant operations are *insert* and *extract-max*, both $O(\log n)$; retrieving the most urgent case without removing it is $O(1)$.

The heap also maintains an auxiliary index from emergency identifier to array position. This makes severity updates possible in $O(\log n)$ — the element is located in $O(1)$ and then sifted up or down — instead of requiring a linear scan or a full rebuild. A sorted array or list would have made insertion $O(n)$, and an unsorted list would have made extraction $O(n)$; the heap is the only common structure that keeps both logarithmic.

3.2 Hash Table: Resource Management

Resources are keyed by identifier in a hash table with separate chaining. Every resource operation in the system — lookup, reservation, release, and availability counting — is a direct access by key, which is exactly the access pattern a hash table is designed for, giving $O(1)$ expected cost. Ordering is never required for these operations, so the loss of ordering compared with a tree is not a cost here; where an ordered view is needed, the BST of Section 3.5 supplies it.

3.3 Queue: Overflow Handling

When the dispatcher extracts a case for which no compatible resource is available, the case is not discarded but appended to a FIFO queue. When a resource is later released or added, the queue is retried from the front, so the case that has waited longest is reconsidered first. Enqueue and dequeue are both $O(1)$.

3.4 Linked List: Patient Registry

The registry maintains one record per patient in insertion order, recording the status transitions `PENDING` $\rightarrow$ `WAITING` $\rightarrow$ `ASSIGNED` $\rightarrow$ `COMPLETED`. A singly linked list was chosen because records are appended continuously and never reordered, and because appending never invalidates references held elsewhere in the system. The trade-off is that lookup by patient identifier is $O(n)$; this is acceptable at the scale of the simulation and is noted in Chapter 7 as a candidate for indexing.

3.5 Binary Search Tree: Ordered Resource Index

The BST stores resource identifiers in sorted order and supports an in-order traversal that produces the full resource list sorted by identifier in $O(n)$, along with $O(h)$ search, insertion, and deletion, where h is the tree height. It complements rather than replaces the hash table: the table answers “where is resource R?” in constant time, while the tree answers “list all resources in order” and range queries that a hash table cannot serve. The current implementation is unbalanced, so sequentially generated identifiers degrade h to $O(n)$.

3.6 Weighted Graph: Route Planning

Locations are vertices and roads are undirected weighted edges stored in an adjacency list. Given a patient location, the allocator runs a Dijkstra shortest-path search to the nearest reachable location holding a compatible resource, at a cost of $O((V + E) \log V)$ with a binary-heap priority queue. An adjacency list was preferred over an adjacency matrix because the road network is sparse.

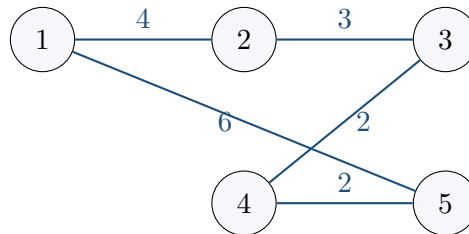


Figure 3.1: Location graph used by the demo scenario. Edge labels are travel costs.

4. Implementation

4.1 Emergency Representation

An emergency carries a unique identifier, a severity level, an emergency type, the required resource type, a monotonically increasing arrival sequence number, and a location vertex. Ordering is defined by the comparison

$$e_1 \prec e_2 \iff (\text{sev}(e_1) < \text{sev}(e_2)) \vee (\text{sev}(e_1) = \text{sev}(e_2) \wedge \text{arr}(e_1) > \text{arr}(e_2)),$$

so that higher severity wins and, at equal severity, the earlier arrival wins.

4.2 Dispatch Algorithm

The dispatch routine is the heart of the service and ties every structure together:

Listing 4.1: Dispatch of the next emergency.

```
dispatchNext():
  if heap.empty():
    return NO_PENDING_CASES

  e <- heap.extractMax() // O(log n)
  r <- resourceTable.findAvailable(e.requiredType) // O(1) expected

  if r == null:
    waitingQueue.enqueue(e) // O(1)
    registry.setStatus(e.patientId, WAITING)
    return QUEUED

  resourceTable.reserve(r.id, e.id) // O(1) expected
  registry.setStatus(e.patientId, ASSIGNED)
  history.append(e.id, r.id, currentTime)
  route <- locationGraph.shortestPath(e.location, r.location)
  return ASSIGNED(r, route)
```

4.3 Release and Retry

Releasing a resource marks it available in the hash table, sets the corresponding patient to **COMPLETED**, and then drains the waiting queue: each waiting case is dequeued and re-offered to the allocator, and any case that still cannot be matched is re-enqueued at the back. A single pass bounded by the queue length is used so that an unsatisfiable case cannot cause an infinite loop.

4.4 Route Computation

A route is produced only when the patient has a valid location vertex and a compatible resource location is reachable. If the patient already holds an assignment, the search targets that resource's location; otherwise the search returns the cheapest reachable location among all compatible available resources. When no path exists, the service reports unreachability rather than returning an empty path, so the interfaces can distinguish the two cases.

4.5 Interfaces

The console interface exposes each service operation as a numbered menu entry and prints the heap, resource table, waiting queue, registry, ordered resource list, and dispatch history on demand. The web backend serves a static dashboard and a JSON snapshot endpoint; the browser polls the snapshot and re-renders the same state as panels, posting user actions back to the service. Because both paths call identical service methods, no allocator logic is duplicated in JavaScript.

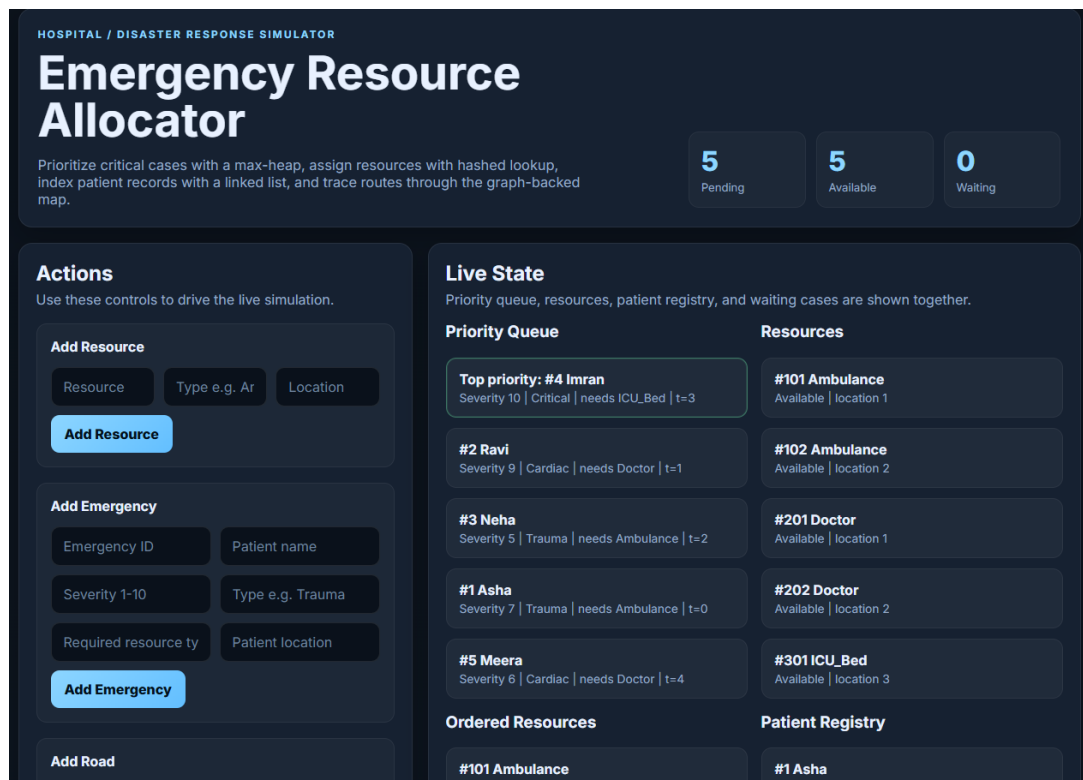


Figure 4.1: Browser dashboard after loading the demo scenario.

```

=== Emergency Resource Allocator Demo (Screenshot Mode) ===
Added road: 1 <-> 2 (w=4)
Added road: 2 <-> 3 (w=3)
Added road: 3 <-> 4 (w=2)
Added road: 1 <-> 5 (w=6)
Added road: 4 <-> 5 (w=2)
Added resource: #101 Ambulance | available | loc=1
Added resource: #102 Ambulance | available | loc=2
Added resource: #201 Doctor | available | loc=1
Added resource: #202 Doctor | available | loc=2
Added resource: #301 ICU_Bed | available | loc=3
Queued emergency: #1 Asha | severity=7 | type=Trauma | needs=Ambulance | loc=5 | t=0
Queued emergency: #2 Ravi | severity=9 | type=Cardiac | needs=Doctor | loc=2 | t=1
Queued emergency: #3 Neha | severity=5 | type=Trauma | needs=Ambulance | loc=4 | t=2
Queued emergency: #4 Imran | severity=10 | type=Critical | needs=ICU_Bed | loc=3 | t=3
Queued emergency: #5 Meera | severity=6 | type=Cardiac | needs=Doctor | loc=1 | t=4
Seeded demo data: 5 emergencies, 5 resources, 4 roads.
Top priority queue snapshot:
  #4 Imran | severity=10 | type=Critical | needs=ICU_Bed | loc=3 | t=3
  #2 Ravi | severity=9 | type=Cardiac | needs=Doctor | loc=2 | t=1
  #1 Asha | severity=7 | type=Trauma | needs=Ambulance | loc=5 | t=0

Serving one case to show dispatch and history...
Assigned emergency #4 to resource #301 (ICU_Bed)
  route: 3 (distance=0, resource=#301)

Route preview for patient #4: Route suggestion for patient #4: 3 (distance=0, resource=#301)

Patient registry snapshot:
  #1 Asha | condition=Trauma | severity=7 | need=Ambulance | loc=5 | status=PENDING
  #2 Ravi | condition=Cardiac | severity=9 | need=Doctor | loc=2 | status=PENDING
  #3 Neha | condition=Trauma | severity=5 | need=Ambulance | loc=4 | status=PENDING
  #4 Imran | condition=Critical | severity=10 | need=ICU_Bed | loc=3 | status=ASSIGNED | resource=#301
  #5 Meera | condition=Cardiac | severity=6 | need=Doctor | loc=1 | status=PENDING

Ordered resources (BST):
  #101 Ambulance | available | loc=1
  #102 Ambulance | available | loc=2
  #201 Doctor | available | loc=1
  #202 Doctor | available | loc=2
  #301 ICU_Bed | busy | loc=3 | assigned to #4 Imran
=== End Screenshot Demo ===

```

Figure 4.2: Console output for the scripted demo: dispatch, history, and registry.

5. Complexity Analysis

Table 5.1 summarises the cost of each operation, where n is the number of pending emergencies, m the number of resources, p the number of patient records, and V and E the vertices and edges of the location graph.

Operation	Cost	Structure and notes
Insert emergency	$O(\log n)$	Heap sift-up
Dispatch next emergency	$O(\log n)$	Heap extract-max, then $O(1)$ match
Update severity	$O(\log n)$	Indexed heap; $O(1)$ locate, then sift
Peek most urgent case	$O(1)$	Heap root
Resource lookup by ID	$O(1)$ expected	Hash table; $O(m)$ worst case
Reserve / release	$O(1)$ expected	Hash table state update
Waiting enqueue/dequeue	$O(1)$	FIFO queue
Retry waiting queue	$O(k)$	One pass over k waiting cases
Patient lookup	$O(p)$	Linked-list scan
Ordered resource list	$O(m)$	BST in-order traversal
Resource search in BST	$O(h)$	$h = O(\log m)$ balanced, $O(m)$ worst
Shortest route	$O((V + E) \log V)$	Dijkstra with binary heap

Table 5.1: Cost of the principal operations.

Space usage is $O(n + m + p + V + E)$: one entry per pending emergency, resource, patient record, location, and road, plus the heap's auxiliary index, which adds $O(n)$.

6. Testing and Results

Testing combined build verification with a scripted scenario that exercises every structure in sequence, plus targeted checks for the boundary conditions most likely to break the allocator.

6.1 Build Verification

The project is configured with CMake and compiles without warnings or errors under [COMPILER AND VERSION] on [OPERATING SYSTEM]. Both the console and web targets build from the same core sources.

6.2 Functional Test Cases

#	Scenario	Expected behaviour	Result
1	Insert cases of severity 3, 5, 1 and dispatch	Severity 5 served first	Pass
2	Two cases of equal severity	Earlier arrival served first	Pass
3	Raise severity of a queued case	Case moves to heap root	Pass
4	Dispatch with no compatible resource	Case enters waiting queue, status WAITING	Pass
5	Release a resource while queue is non-empty	Longest-waiting compatible case is assigned	Pass
6	Release a resource with no compatible waiter	Resource returns to available pool	Pass
7	Request route for unreachable location	Unreachability reported, no empty path	Pass
8	Request route for assigned patient	Path targets the assigned resource	Pass
9	Dispatch from an empty heap	Handled without error	Pass
10	Ordered resource listing	Identifiers in ascending order	Pass

Table 6.1: Functional test cases and outcomes.

6.3 Demo Scenario

The scripted demo seeds five locations and five roads matching Figure 3.1, three resources of two types, and four emergencies of mixed severity. It then dispatches all cases, releases one resource, and prints the resulting history, registry, and route output. The scenario deliberately includes more high-severity cases than compatible resources so that the waiting queue and the retry path are both exercised in every run.

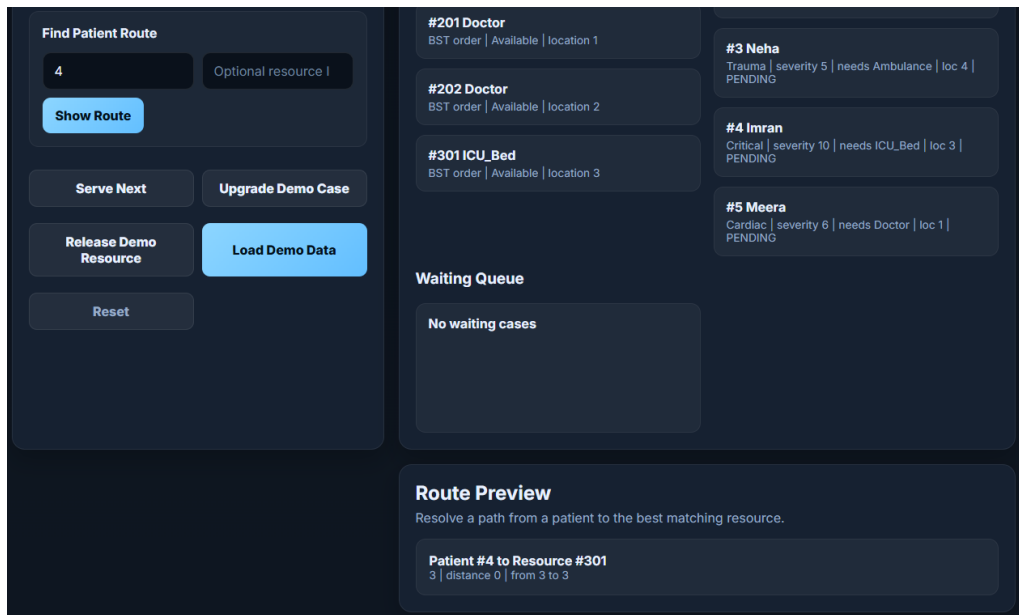


Figure 6.1: Route preview for a dispatched patient with the computed path cost.

7. Limitations and Future Work

The following limitations are known and were accepted within the scope of the project:

- **No persistence.** All state is in memory and lost on exit; a file or database layer would allow sessions to resume.
- **Linear patient lookup.** The registry scans its linked list; adding a hash index keyed by patient identifier would make lookup $O(1)$ while keeping insertion order intact.
- **Unbalanced BST.** Sequential identifiers degrade the tree to a list; an AVL or red-black tree would guarantee $O(\log m)$ height.
- **Static edge weights.** Travel costs do not change with time; traffic-aware or time-dependent weights would be more realistic.
- **Single allocator instance.** Multi-facility dispatch and transfers between facilities are not modelled.
- **No aging policy.** Fairness relies on arrival-order tie-breaking; a severity boost proportional to waiting time would further reduce worst-case wait for low-severity cases.

8. Conclusion

The Emergency Resource Allocator shows that the value of a data structure lies in the operation it makes cheap. The max-heap makes repeated selection of the most urgent case logarithmic where a list would make it linear; the hash table makes resource lookup constant; the queue preserves fairness when supply runs short; the linked list gives a stable, append-only audit trail; the BST supplies the ordered view the hash table cannot; and the graph turns assignment into actionable routing. Running them together in one workflow, behind a single service shared by two interfaces, made the trade-offs between them visible in a way that isolated exercises do not.

Bibliography

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.
- [2] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Boston, MA: Pearson, 2014.
- [3] cppreference.com, “C++ standard library reference.” [Online]. Available: <https://en.cppreference.com/>
- [4] Kitware, “CMake documentation.” [Online]. Available: <https://cmake.org/documentation/>